

Laboratorio: recursión y backtracking

MOMENTO 1 — Enunciado del Laboratorio 3

El caso: rutas de recolección en un barrio. Tienen un mapa como matriz, donde cero es calle transitable, uno es calle bloqueada, y hay puntos de acopio marcados con dos. El camión entra por la esquina superior izquierda.

- **Parte A — El camino básico (35%)**

Encuentren un camino desde la entrada hasta la esquina inferior derecha, esquivando los bloqueos, e imprima el mapa con el camino marcado. Este es el laberinto del lunes, adaptado.

- **Parte B — Recolección obligatoria (40%)**

Ahora el que hace pensar. El camión no puede simplemente llegar a la salida: tiene que pasar por todos los puntos de acopio antes de salir. Encuentren un camino que los recoja a todos.

Piénsenlo antes de programar. El estado ya no es solo dónde estoy: es dónde estoy y cuántos puntos llevo recogidos. El caso base cambia: llegar a la salida ya no basta, hay que llegar con todos recogidos.

- **Parte C — Análisis (25%)**

En el archivo de respuestas: cuántos caminos distintos encontró su algoritmo en la Parte A; qué pasa con el tiempo de ejecución cuando el mapa crece de cinco por cinco a diez por diez, medido de verdad, no estimado; y una explicación en cuatro líneas de por qué este problema no se puede resolver bien con ciclos anidados.

Como siempre, los dos lenguajes. Y en C++ compilen con el sanitizer, aunque hoy no haya memoria dinámica: una recursión desbordada también la detecta.

MOMENTO 2 — Desarrollo

- **PARTE A** Si un equipo dice «se cuelga», casi siempre es que no está marcando las casillas visitadas, y da vueltas en círculo. Si dice «no encuentra salida cuando sí hay», es que no está desmarcando.
- **PARTE B** El atasco es de diseño: intentan resolverlo con la misma función de la Parte A. La pregunta que desatasca es «¿qué información necesita llevar la función además de la posición?».
- **PARTE C** Recuérdeles medir de verdad, con la librería de tiempo, no estimar. Es la primera vez que miden y les cuesta.

Antes de arrancar, dos consejos. Uno: empiecen imprimiendo el mapa en cada llamada recursiva, aunque ensucie la salida. Ver por dónde va el algoritmo es la forma más rápida de encontrar el error, y después borran los prints.

Y dos: prueben primero con un mapa de tres por tres que hayan resuelto a mano. Si el algoritmo falla ahí, no van a poder depurarlo en uno de diez por diez.

3. ANÁLISIS

GRUPO : Sergio Rios - Julian Hernandez - Edward Castillo

Parte C – Análisis

MOMENTO 1: Parte A y B

Configuración del Entorno y Convenciones

Para el desarrollo de este modelo, se ha estructurado una representación matricial del espacio de trabajo. Con el fin de facilitar la lectura y el análisis de las rutas del camión recolector, se establecen las siguientes convenciones:

- **0 (Libre):** Espacios transitables y libres de obstáculos.
- **1 (Muro):** Zonas bloqueadas o barreras físicas intransitables.
- **2 (Punto de Acopio):** Estaciones estratégicas designadas para la recolección y recogida de materiales por parte del camión.

```
# MATRIZ 4 X 4 (Prueba básica)
Ruta_barrio = [
    [0, 0, 1, 2],
    [2, 0, 0, 0],
    [0, 0, 1, 0],
    [1, 0, 0, 2],
]

# AMPLITUD - MATRIZ 5 X 5 (Prueba de escalabilidad Parte C)
# Ruta_barrio = [
#     [0, 0, 1, 0, 2],
#     [1, 0, 1, 0, 0],
#     [0, 0, 0, 0, 0],
#     [0, 1, 1, 1, 2],
#     [2, 0, 0, 1, 0],
# ]

# MATRIZ 10 X 10 VÁLIDA (Con múltiples caminos y puntos de acopio)
# 0 = calle libre, 1 = muro/bloqueo, 2 = punto de acopio
# Ruta_barrio = [
#     [0, 0, 0, 1, 0, 0, 2, 0, 0, 0],
#     [1, 1, 0, 1, 0, 1, 1, 1, 1, 0],
#     [2, 0, 0, 0, 0, 0, 0, 0, 0, 0],
#     [0, 1, 1, 1, 1, 1, 0, 1, 1, 0],
#     [0, 0, 2, 0, 0, 0, 0, 1, 2, 0],
#     [1, 1, 0, 1, 1, 1, 0, 1, 0, 0],
#     [0, 0, 0, 0, 2, 0, 0, 0, 0, 1],
#     [0, 1, 1, 1, 0, 1, 1, 1, 0, 0],
#     [0, 0, 0, 1, 0, 0, 0, 1, 1, 0],
#     [1, 1, 0, 0, 0, 2, 0, 0, 0, 0],
# ]
```

Evaluación de Rutas y Complejidad

Con el fin de identificar las rutas óptimas de recolección y salida, se implementó un contador de tiempo de procesamiento. Los resultados demuestran una relación directa: el incremento en las dimensiones de la matriz impacta negativamente en el rendimiento, aumentando de manera notable el tiempo de ejecución del sistema a mayor escala.

```
Hay salida. Primer camino encontrado:
R . # P
R . . .
R R # .
# R R R

Camino distintos encontrados: 1
Tiempo de ejecución real: 0.00003130 segundos

Hay salida. Primer camino encontrado:
R R # . P
# R # . .
. R R R R
. # # # R
P . . # R

Camino distintos encontrados: 1
Tiempo de ejecución real: 0.0004670 segundos

Hay salida. Primer camino encontrado:
R R R # . . P . . .
# # R # . # # # .
P . R R R R . . .
. # # # # R # # .
. . P . . R # P .
# # . # # # R # . .
. . . . P . R R R #
. # # # . # # # R R
. . . # . . . # # R
# # . . . P . . . R

Camino distintos encontrados: 1
Tiempo de ejecución real: 0.00049400 segundos
```

MOMENTO 2: Lógica de Exploración y Evolución del Algoritmo

Parte A: Arquitectura Base del Algoritmo de Búsqueda

El núcleo computacional del proyecto descansa sobre un motor recursivo asistido por la técnica de *backtracking*. Su funcionamiento se rige por un ciclo secuencial de seis etapas invariables:

1. **Control de límites:** ¿La posición actual excede los márgenes de la matriz? $\rightarrow$ *Movimiento inválido.*
2. **Validación de obstáculo/visita:** ¿Es un muro o una celda previamente transitada? $\rightarrow$ *Movimiento inválido.*

3. **Registro de tránsito:** Se marca la casilla y se anexa de inmediato a la ruta activa.
4. **Evaluación de meta (Caso Base):** ¿Se alcanzó la salida? $\rightarrow$ *Se almacena la solución.*
5. **Ramificación (Caso Recursivo):** Se despliegan los movimientos posibles bajo un orden jerárquico estricto: **abajo, derecha, arriba e izquierda.**
6. **Retroceso operativo (*Backtracking*):** Se desmarca la celda y el proceso retorna a la instancia anterior.

Fundamentos del Control de Estados

La ejecución de los pasos 3 y 6 cumple un rol crítico: marcar la celda previene la formación de bucles infinitos, mientras que desmarcarla garantiza que un intento fallido no clausure de manera permanente rutas alternativas viables. Omitir esta reversión provocaría falsos negativos, reportando la inexistencia de rutas en escenarios donde sí las hay.

Nota de diseño: Esta estructura constituyó la base fundacional del software. La **Parte A** contempla el escenario estándar (sin la obligación de recolectar todos los puntos de acopio), sirviendo como punto de partida para la **Parte B**.

Parte B: Restricciones Avanzadas y Gestión de Acopios

Para esta fase, cruzar la línea de meta ya no valida por sí solo una solución. El criterio de aceptación exige el cumplimiento dual y simultáneo de dos condiciones: **haber visitado la salida y haber recogido la totalidad de los puntos de acopio.**

Para soportar esta exigencia, la función recursiva se expandió, integrando un mecanismo para rastrear el estado de recolección de cada rama de manera independiente.

Variables Estructurales y de Control

- **TOTAL_ACOPIO:** Cantidad neta de puntos de recolección presentes en el entorno. Se calcula mediante un escaneo automatizado de la matriz, eximiendo al usuario de la introducción manual de coordenadas.
- **recolectados:** Contador dinámico del progreso de recolección exclusivo de la rama actual, propagado a través de los parámetros de la llamada recursiva.
- **Matriz de rastreo camino[f][c]:** Evoluciona más allá de un booleano tradicional. Ahora almacena el valor numérico **recolectados + 1**, permitiendo que el sistema reconozca con cuántos acopios previos se arribó a una celda específica.

Criterios de Validación y Dinámica de Puntos

- **Actualización del Caso Base:**
Una ruta se consolida formalmente como solución únicamente si confluyen la posición de salida y el agotamiento total de los acopios del mapa.
- **Interacción con los Puntos de Acopio:**
Al interceptar un punto de acopio (2), su valor cambia de manera transitoria a 0 (espacio libre). Al activarse el *backtracking* y replegarse la función, la celda recupera su valor original (2).
Esta dinámica impide que un mismo punto sea contabilizado por duplicado dentro de una

misma trayectoria y asegura que la condición de **TOTAL_ACOPIO** responda a una cobertura real y efectiva del terreno.

Análisis de Fallos, Optimización y Estrategia de Poda

1. Problema Encontrado Durante las Pruebas

Durante la fase de pruebas experimentales, se identificó una limitación crítica en la primera versión del programa. En su diseño inicial, cada casilla visitada se marcaba directamente con un **1**, lo cual impedía que el algoritmo volviera a transitar por una celda previamente recorrida.

Este enfoque funcionaba de manera correcta siempre y cuando todos los puntos de acopio pudieran alcanzarse mediante trayectorias directas y sin necesidad de retraso. Sin embargo, el fallo se evidenció al introducir un punto de acopio en una calle sin salida —específicamente en la posición **(4,0)**—, donde el camión debía ingresar y obligatoriamente devolverse por el mismo camino para salir.

Comparativa de Mapas de Prueba

Mapa que funcionaba correctamente:

```
0 2 1 0 0
1 0 1 0 1
0 0 2 0 2
0 1 1 1 0
0 0 0 1 0
```

Mapa que inicialmente fallaba (con punto de acopio en (4,0)):

```
2 0 0 1 0 <-- (Modificación en la fila inferior)
1 0 1 0 1
0 0 2 0 2
0 1 1 1 0
2 0 0 1 0
```

En este segundo escenario, el sistema informaba erróneamente que no existía ningún recorrido válido, a pesar de que la solución física era factible

2. Diagnóstico y Corrección del Algoritmo

Tras un análisis exhaustivo del comportamiento del sistema, se determinó que la falla no residía en la estructura de recursión ni en el *backtracking*, sino en la **pobreza de información de la marca de visitantes**.

- **El problema:** Marcar una celda de forma estática eliminaba la posibilidad de volver a visitarla, ignorando que una misma coordenada representa estados lógicos completamente distintos si se arriba a ella con un acumulado diferente de puntos de acopio.
- **La solución implementada:** Se rediseñó la lógica de casillas visitadas para que almacene un estado compuesto (la posición actual junto con la cantidad de puntos de acopio acumulados hasta ese instante).
 - Un movimiento solo se descarta si el algoritmo **ya ha estado en esa misma casilla con exactamente la misma cantidad de puntos recolectados**.
 - Si se llega a la celda con un conteo distinto, la ruta continúa por tratarse de un estado único de búsqueda.
- **Ajuste en el Backtracking:** Se eliminó la restauración plana a 0 durante el retroceso; en su lugar, se recupera de forma precisa el valor previo que poseía la casilla antes de la llamada recursiva, resguardando la integridad de cada rama.

Gracias a estas modificaciones, el algoritmo superó el fallo y resolvió exitosamente el mapa problemático, hallando una ruta óptima de **14 pasos**.

3. Optimización: Ruta Óptima y Estrategia de Poda

Para potenciar el rendimiento frente al crecimiento exponencial de los mapas, se ajustó el comportamiento global del sistema:

- **Búsqueda exhaustiva flexible (No detención en Caso Base):** El caso base ya no interrumpe la exploración; su función actual es registrar la solución encontrada y permitir que el *backtracking* continúe explorando, guardando una copia únicamente cuando el trayecto es estrictamente más corto que el récord previo.
- **Implementación de Poda de Rendimiento:** Dado que evaluar la totalidad de las ramas resultaba computacionalmente inviable en mapas abiertos, se añadió una condición de corte: *si la longitud del recorrido en construcción iguala o supera los pasos de la mejor ruta conocida, la rama se descarta de inmediato* al comprobarse que ninguna continuación mejorará el resultado.
- **Garantía de Optimalidad:** Al descartar exclusivamente caminos que ya son subóptimos o iguales a los mejores encontrados, la ruta óptima jamás se ve excluida. Como contrapartida a esta optimización, el programa prioriza la eficiencia y deja de contabilizar la cantidad total de caminos posibles.

1. Caminos distintos encontrados en la Parte A

Probamos con ocho matrices de 5×5 y la de 10×10 . La cantidad de caminos depende mucho más de las casillas libres conectadas que del tamaño de la matriz:

Matriz	M1	M2	M3	M4	M5	M6	M7	M9 (10×10)
Casillas libres	18	18	19	17	19	21	22	66
Caminos (Parte A)	2	2	7	1	8	72	104	183

2. Tiempo de ejecución al pasar de 5×5 a 10×10

Medido con `time.perf_counter()` encerrando solo la llamada recursiva, sin la impresión de resultados. Las mediciones de la Parte A se promediaron sobre varias repeticiones por ser de microsegundos.

Matriz	Tamaño	Parte A	Parte B (ruta óptima)	Llamadas recursivas (Parte B)
M1 – M5, M8	5×5	0,005 – 0,05 ms	menos de 0,001 s	81 – 609
M6	5×5	0,26 ms	0,001 s	4.105
M7	5×5	0,40 ms	0,04 s	156.269
M9	10×10	1,19 ms	unos 30 s	97.157.941

En la Parte A el tiempo crece unas tres veces. En la Parte B pasa de centésimas de segundo a unos treinta segundos y las llamadas recursivas se multiplican por más de seiscientas. El tiempo no depende del tamaño de la matriz sino del número de recorridos que hay que considerar, y ese número crece de forma exponencial con las casillas libres conectadas entre sí: duplicar el lado no duplica el trabajo, lo multiplica por varios órdenes de magnitud. Sin la poda, la matriz de 10×10 no termina. El tiempo varía entre corridas según la carga del equipo; las llamadas recursivas son siempre las mismas para una matriz dada.

3. ¿Por qué no se resuelve bien con ciclos anidados?

Un ciclo anidado tiene un número de niveles fijo, escrito en el código, mientras que aquí la cantidad de decisiones encadenadas depende del mapa y no se conoce al programar.

Además, cada decisión modifica el estado del recorrido —casillas marcadas y puntos recogidos— y ese estado debe deshacerse al retroceder, algo que los ciclos no hacen por sí solos.

La recursión resuelve las dos cosas: la pila de llamadas guarda el recorrido actual y su profundidad se adapta a cada mapa.

Para hacerlo con ciclos habría que construir a mano una pila explícita con las posiciones pendientes, es decir, implementar la recursión.